

Билет 21. Модель памяти C++. `sequenced-before`, `happens-before`, `synchronizes-with`, `memory_order`, барьеры памяти, `mfence`

0. Формулировка билета

Модель памяти C++. Понятия `sequenced-before`, `happens-before`, `synchronizes-with`. Разновидности `memory_order`. Барьеры памяти, инструкция `mfence`.

Главная идея:

Модель памяти C++ описывает, какие операции с памятью могут быть видны другим потокам, какие переупорядочивания допустимы, что считается data race и какими средствами можно установить порядок между действиями разных потоков.

В однопоточном коде обычно кажется, что всё просто:

```
x = 1;  
y = 2;
```

Мы думаем:

```
сначала x = 1  
потом y = 2
```

Но в многопоточном коде всё сложнее:

компилятор может переупорядочивать инструкции
процессор может переупорядочивать операции
у ядер есть store buffer
у разных потоков нет общего "очевидного" порядка всех операций

Поэтому нужен формальный язык:

```
sequenced-before  
synchronizes-with  
happens-before  
memory_order_relaxed  
memory_order_acquire  
memory_order_release  
memory_order_acq_rel
```

```
memory_order_seq_cst
fences
```

1. Почему нельзя думать только через “инструкции выполняются по порядку”

В одном потоке компилятор и процессор могут менять порядок операций, если с точки зрения этого же потока поведение не меняется.

Например:

```
int a = x;
int b = y;
```

На уровне оптимизаций порядок загрузок может быть не таким, как в исходном коде.

В однопоточном коде это обычно незаметно.

Но в многопоточном коде другой поток может наблюдать память иначе.

Пример:

```
// thread 1
data = 42;
ready = true;

// thread 2
if (ready) {
    use(data);
}
```

Интуитивно хочется думать:

```
если ready == true, значит data уже 42
```

Но без синхронизации это неверно.

Если `data` и `ready` — обычные неатомарные переменные, здесь вообще data race.

Если `ready` — atomic, но с неправильным memory order, поток может увидеть `ready == true`, но не получить гарантию видимости записи `data = 42`.

2. Data race

В C++ data race возникает, если:

два или более потока обращаются к одной и той же области памяти
хотя бы одно обращение — запись
операции не упорядочены через happens-before
и это не атомарные операции

Data race в C++ — это undefined behavior.

Пример:

```
int counter = 0;

void worker() {
    ++counter;
}
```

Если два потока одновременно вызывают `worker`, то:

оба обращаются к counter
есть запись
нет синхронизации
counter не atomic

Это data race.

Важно:

Data race — это не просто “неправильный ответ”.
В C++ это undefined behavior.

Решения:

```
std::mutex m;
int counter = 0;

void worker() {
    std::lock_guard<std::mutex> lock(m);
    ++counter;
}
```

или:

```
std::atomic<int> counter{0};

void worker() {
    counter.fetch_add(1);
}
```

3. Race condition vs data race

Эти понятия лучше различать.

Data race

Это формальное нарушение модели памяти C++.

Пример:

```
int x = 0;

thread 1:
    x = 1;

thread 2:
    x = 2;
```

Без синхронизации это data race.

Race condition

Это более общее логическое понятие.

Программа может быть без data race, но с race condition.

Например:

```
std::atomic<int> balance{100};

void withdraw_100() {
    if (balance.load() >= 100) {
        balance.store(balance.load() - 100);
    }
}
```

Data race нет, потому что `balance` atomic.

Но логическая гонка есть: два потока могут оба увидеть `100` и оба снять деньги неправильно.

То есть:

```
data race:  
    нарушение правил доступа к памяти  
  
race condition:  
    результат зависит от нежелательного порядка событий
```

4. `sequenced-before`

`sequenced-before` — отношение порядка внутри одного потока.

Если в одном потоке написано:

```
int x = 1;  
int y = 2;
```

то обычно вычисление первого выражения `sequenced-before` второго.

Упрощённо:

`sequenced-before` описывает порядок действий внутри одного потока с точки зрения абстрактной машины C++.

Пример:

```
a = 1;  
b = 2;
```

Внутри потока:

```
a = 1 sequenced-before b = 2
```

Важно:

`sequenced-before` само по себе не создаёт синхронизацию с другим потоком.

Оно говорит:

в этом потоке действие А раньше действия В

Но другой поток узнает об этом порядке только если есть межпоточная синхронизация.

5. synchronizes-with

`synchronizes-with` — отношение между действиями в разных потоках.

Оно возникает, например, между:

release operation в одном потоке
и acquire operation в другом потоке,
если acquire прочитал значение, записанное release

Классический пример:

```
int data = 0;
std::atomic<bool> ready{false};

// thread 1
data = 42;
ready.store(true, std::memory_order_release);

// thread 2
if (ready.load(std::memory_order_acquire)) {
    // здесь гарантированно видно data = 42
    use(data);
}
```

Если `load(acquire)` прочитал `true`, записанный `store(release)`, то:

`ready.store(true, release)`
 `synchronizes-with`
`ready.load(acquire)`

И благодаря этому все записи до `release` в `thread 1` становятся видимы после `acquire` в `thread 2`.

6. happens-before

`happens-before` — главное отношение порядка в модели памяти C++.

Оно строится из:

sequenced-before
synchronizes-with
транзитивности

Упрощённо:

если A sequenced-before B,
то A happens-before B

если A synchronizes-with B,
то A happens-before B

если A happens-before B и B happens-before C,
то A happens-before C

Главная идея:

Если запись happens-before чтения, то чтение должно видеть эту запись или более позднюю запись в допустимом порядке.

И ещё важнее:

Если конфликтующие неатомарные операции не упорядочены через happens-before, получается data race.

7. Пример happens-before через release/acquire

Код:

```
int data = 0;
std::atomic<bool> ready{false};

// thread 1
data = 42;
ready.store(true, std::memory_order_release);

// thread 2
while (!ready.load(std::memory_order_acquire)) {
}
assert(data == 42);
```

Разберём порядок:

В thread 1:

```
data = 42
    sequenced-before
ready.store(true, release)
```

Между потоками:

```
ready.store(true, release)
    synchronizes-with
ready.load(acquire), который прочитал true
```

В thread 2:

```
ready.load(acquire)
    sequenced-before
чтение data
```

По транзитивности:

```
data = 42
    happens-before
чтение data во втором потоке
```

Поэтому второй поток после acquire-load видит `data = 42`.

8. Mutex и happens-before

Mutex тоже создаёт синхронизацию.

Если один поток делает:

```
m.unlock();
```

а другой потом успешно делает:

```
m.lock();
```

на тот же mutex, то:

```
unlock synchronizes-with subsequent lock
```

Пример:


```
std::mutex m;
int x = 0;

// thread 1
{
    std::lock_guard<std::mutex> lock(m);
    x = 42;
}

// thread 2
{
    std::lock_guard<std::mutex> lock(m);
    assert(x == 42);
}
```

Если thread 2 вошёл в критическую секцию после thread 1, то запись `x = 42` видна.

Упрощённо:

```
mutex unlock – release
mutex lock   – acquire
```

9. Атомики

`std::atomic<T>` даёт операции без data race.

Например:

```
std::atomic<int> x{0};

thread 1:
    x.store(1);

thread 2:
    int r = x.load();
```

Data race нет.

Но вопрос:

Какие ещё гарантии порядка даёт эта atomic-операция?

Это зависит от memory order.

По умолчанию атомарные операции используют:

```
std::memory_order_seq_cst
```

Это самый сильный порядок.

Но можно явно указать более слабый:

```
x.store(1, std::memory_order_relaxed);  
x.load(std::memory_order_relaxed);
```

10. `memory_order_relaxed`

`memory_order_relaxed` даёт только атомарность операции над конкретной `atomic`-переменной.

Он не создаёт синхронизацию между потоками.

Пример:

```
std::atomic<int> counter{0};  
  
void worker() {  
    counter.fetch_add(1, std::memory_order_relaxed);  
}
```

Это нормально для счётчика статистики, если нам нужно только:

```
не потерять инкременты  
не получить data race
```

Но `relaxed` не гарантирует порядок с другими переменными.

Пример, где `relaxed` недостаточен:

```
int data = 0;  
std::atomic<bool> ready{false};  
  
// thread 1  
data = 42;  
ready.store(true, std::memory_order_relaxed);  
  
// thread 2  
if (ready.load(std::memory_order_relaxed)) {
```

```
use(data);  
}
```

Даже если thread 2 увидел `ready == true`, нет гарантии, что он корректно увидит `data = 42`.

Формула:

```
relaxed:  
    атомарность да  
    порядок между потоками нет
```

11. `memory_order_release`

`memory_order_release` используется на записи.

Например:

```
ready.store(true, std::memory_order_release);
```

Release запрещает переупорядочивать предыдущие записи после `release-store` в смысле видимости для потока, который сделает `acquire`.

Идея:

```
всё, что было до release в этом потоке,  
должно стать видимым потоку,  
который увидит этот release через acquire
```

Пример:

```
data = 42;  
ready.store(true, std::memory_order_release);
```

Release как бы публикует данные.

12. `memory_order_acquire`

`memory_order_acquire` используется на чтении.

Например:

```
if (ready.load(std::memory_order_acquire)) {  
    use(data);  
}
```

Acquire запрещает последующим операциям “уехать” до acquire-load в смысле видимости.

Если acquire-load прочитал значение, записанное release-store, то возникает `synchronizes-with`.

Связка:

```
release-store  
acquire-load, прочитавший это значение
```

создаёт межпоточную синхронизацию.

Формула:

```
release:  
    публикует всё, что было до него  
  
acquire:  
    принимает публикацию и видит всё, что было до release
```

13. Release/acquire: главный паттерн

Главный паттерн:

```
int data = 0;  
std::atomic<bool> ready{false};  
  
// producer  
data = 42;  
ready.store(true, std::memory_order_release);  
  
// consumer  
while (!ready.load(std::memory_order_acquire)) {  
}  
assert(data == 42);
```

Если consumer увидел `true`, то:

```
data = 42 happens-before чтение data в consumer
```

Потому что:

```
data = 42
sequenced-before
ready.store(true, release)
synchronizes-with
ready.load(acquire)
sequenced-before
чтение data
```

Это один из главных примеров для билета.

14. `memory_order_acq_rel`

`memory_order_acq_rel` используется для операций, которые одновременно читают и пишут.

Например, RMW-операции:

```
x.fetch_add(1, std::memory_order_acq_rel);
x.compare_exchange_weak(expected, desired,
                        std::memory_order_acq_rel,
                        std::memory_order_relaxed);
```

RMW = read-modify-write.

Примеры:

```
fetch_add
exchange
compare_exchange
```

`acq_rel` означает:

```
acquire-часть:
    операция может принять синхронизацию от предыдущего release

release-часть:
    операция может опубликовать предыдущие действия для последующего acquire
```

Используют, когда операция одновременно:

читает старое состояние
и публикует новое состояние

15. `memory_order_seq_cst`

`memory_order_seq_cst` — sequential consistency.

Это порядок по умолчанию для атомиков.

Он даёт:

атомарность
acquire/release гарантии
единый глобальный порядок всех `seq_cst` операций

Интуитивно:

Все `seq_cst` атомарные операции можно представить как выполненные в одном общем порядке, согласованном с порядком внутри каждого потока.

Это проще для рассуждений, но может быть дороже.

Пример:

```
std::atomic<int> x{0};  
  
x.store(1); // seq_cst по умолчанию  
int r = x.load(); // seq_cst по умолчанию
```

На экзамене можно сказать:

`seq_cst` — самый сильный и самый простой для понимания `memory order`

16. `memory_order_consume`

Есть ещё:

`std::memory_order_consume`

Идея была в зависимости по данным:

```
Node* p = head.load(std::memory_order_consume);  
int x = p->value;
```

Но на практике `consume` почти не используют, потому что его сложно корректно реализовать и поддерживать.

Обычно вместо него используют:

```
std::memory_order_acquire
```

На устном достаточно сказать:

`memory_order_consume` практически обычно заменяют на `acquire`.

17. Сводка memory orders

```
relaxed:  
    только атомарность, без синхронизации  
  
release:  
    запись-публикация предыдущих действий  
  
acquire:  
    чтение-принятие публикации  
  
acq_rel:  
    для read-modify-write операций, одновременно acquire и release  
  
seq_cst:  
    самый сильный порядок, плюс единый глобальный порядок seq_cst операций  
  
consume:  
    зависимость по данным, практически редко используется
```

18. Release sequence

Это более тонкое понятие, но его полезно понимать.

Release sequence возникает после `release`-записи в `atomic`-переменную и может продолжаться через `RMW`-операции над той же переменной.

Пример:

```
std::atomic<int> flag{0};
int data = 0;

// thread 1
data = 42;
flag.store(1, std::memory_order_release);

// thread 2
flag.fetch_add(1, std::memory_order_relaxed);

// thread 3
int x = flag.load(std::memory_order_acquire);
```

Если thread 3 acquire-load прочитал значение из release sequence, начатой release-store из thread 1, то синхронизация с thread 1 сохраняется.

Упрощённо:

Release sequence позволяет RMW-операциям продолжать цепочку синхронизации от release-store.

Для базового ответа можно не углубляться, но если спросят — сказать именно так.

19. CAS и memory order success/failure

У `compare_exchange` можно задавать два memory order:

```
compare_exchange_weak(expected, desired,
                      success_order,
                      failure_order);
```

Почему два?

Потому что CAS имеет два исхода.

Success

CAS успешен:

```
прочитал старое значение
записал новое значение
```

Это RMW-операция.

Можно использовать:


```
release
acquire
acq_rel
seq_cst
```

Failure

CAS неуспешен:

```
только прочитал текущее значение
ничего не записал
```

Поэтому failure-order не может быть release или acq_rel, потому что release относится к записи.

Обычно ставят:

```
std::memory_order_relaxed
```

или:

```
std::memory_order_acquire
```

если после неуспешного CAS нужно видеть данные, связанные с прочитанным значением.

Пример:

```
head.compare_exchange_weak(old_head, new_node,
                           std::memory_order_release,
                           std::memory_order_relaxed);
```

20. Store buffer и пример 0, 0

Классический пример, почему интуитивный interleaving не всегда работает.

```
std::atomic<int> x{0};
std::atomic<int> y{0};

int r1 = 0;
int r2 = 0;

// thread 1
x.store(1, std::memory_order_relaxed);
```

```
r1 = y.load(std::memory_order_relaxed);

// thread 2
y.store(1, std::memory_order_relaxed);
r2 = x.load(std::memory_order_relaxed);
```

Возможен результат:

```
r1 == 0
r2 == 0
```

Почему интуитивно странно?

Каждый поток сначала сделал store, потом load.

Но из-за store buffer и relaxed-порядка другой поток может ещё не увидеть запись.

То есть:

```
thread 1 видит y == 0
thread 2 видит x == 0
```

При sequential consistency такой результат был бы невозможен.

Этот пример показывает:

Нельзя всегда думать о многопоточном исполнении как о простом чередовании строк исходного кода.

21. Барьеры памяти

Memory barrier / memory fence — инструкция или операция, которая ограничивает переупорядочивание операций памяти.

Барьеры нужны, чтобы сказать процессору/компилятору:

```
некоторые операции до барьера должны быть завершены/видимы до некоторых
операций после барьера
```

В C++ есть fences:

```
std::atomic_thread_fence(std::memory_order_acquire);
std::atomic_thread_fence(std::memory_order_release);
```

```
std::atomic_thread_fence(std::memory_order_acq_rel);  
std::atomic_thread_fence(std::memory_order_seq_cst);
```

Fences не работают с конкретной atomic-переменной напрямую, но могут участвовать в синхронизации вместе с атомарными операциями.

Для большинства прикладного кода обычно проще использовать acquire/release на самих atomic-операциях.

22. Compiler barrier vs CPU barrier

Есть два уровня.

Compiler barrier

Запрещает компилятору переупорядочивать операции через некоторую точку.

Но не обязательно генерирует аппаратную инструкцию барьера.

CPU memory barrier

Аппаратная инструкция, которая ограничивает переупорядочивание на уровне процессора.

Например на x86 есть:

```
mfence  
sfence  
lfence
```

23. mfence

`mfence` — x86-инструкция full memory fence.

Она гарантирует порядок операций памяти до и после барьера.

Упрощённо:

```
load/store до mfence  
не должны быть переупорядочены с  
load/store после mfence
```

`mfence` дорогая инструкция, потому что ограничивает оптимизации процессора и работу store buffer.

Но важная оговорка:

В обычном C++ коде не нужно руками вставлять `mfence`.
Нужно использовать `std::atomic` и memory order, а компилятор сам сгенерирует нужные инструкции для целевой архитектуры.

На x86 многие acquire/release операции могут быть реализованы без явного `mfence`, потому что x86 уже довольно сильная модель памяти. А вот `seq_cst` fence или некоторые операции могут требовать более тяжёлых инструкций.

24. Связь memory order и железа

C++ memory model — это уровень языка.

Железо — это конкретная архитектура:

```
x86
ARM
RISC-V
```

У разных архитектур разные аппаратные модели памяти.

Например:

```
x86 обычно сильнее
ARM слабее и чаще требует явных барьеров
```

C++ даёт переносимый интерфейс:

```
std::atomic
memory_order_acquire
memory_order_release
memory_order_seq_cst
```

Компилятор отображает это на конкретные инструкции процессора.

Поэтому лучше рассуждать на уровне C++:

```
release/acquire создаёт synchronizes-with
happens-before даёт видимость данных
```

а не на уровне “я точно хочу mfence”.

25. Sequential consistency и SC-DRF

Есть важная идея:

Если программа не имеет data race и использует синхронизацию корректно, то можно рассуждать о ней почти как о sequentially consistent программе.

Это называется SC-DRF:

Sequential Consistency for Data-Race-Free programs

Смысл:

если нет data race,
то программист может думать о выполнении как о некотором чередовании операций потоков,
согласованном с порядком внутри каждого потока

Это главная причина писать программы без data race.

26. Типичные ошибки

Ошибка 1. Думать, что atomic всегда означает “всё синхронизировано”

Нет.

```
std::atomic<bool> ready;
```

может использоваться с `memory_order_relaxed`.

Тогда data race на `ready` нет, но синхронизации с другими данными тоже нет.

Ошибка 2. Использовать relaxed для публикации данных

Плохо:

```
data = 42;  
ready.store(true, std::memory_order_relaxed);
```

Если другой поток по `ready` должен увидеть `data`, нужен release/acquire.

Ошибка 3. Думать, что `volatile` — это синхронизация

`volatile` в C++ не является механизмом межпоточной синхронизации.

Для потоков нужны:

```
std::atomic
mutex
condition_variable
и другие синхронизационные примитивы
```

Ошибка 4. Путать `data race` и `race condition`

Data race — формальное UB.

Race condition — логическая зависимость результата от порядка событий.

Ошибка 5. Использовать `release` на `CAS failure-order`

Failure CAS ничего не записывает, поэтому `release` там не имеет смысла и запрещён.

27. Что сказать коротко

Модель памяти C++ описывает, какие операции с памятью видны другим потокам, какие переупорядочивания допустимы и что считается `data race`. `Data race` возникает, если несколько потоков обращаются к одной неатомарной области памяти, хотя бы одно обращение — запись, и операции не упорядочены через `happens-before`. `Data race` в C++ — `undefined behavior`.

`sequenced-before` — порядок действий внутри одного потока. `synchronizes-with` — межпоточное отношение синхронизации, например `release-store` в одном потоке `synchronizes-with` `acquire-load` в другом, если `acquire` прочитал значение, записанное `release`.

`happens-before` строится из `sequenced-before`, `synchronizes-with` и транзитивности. Если запись `happens-before` чтения, то чтение должно видеть корректно опубликованные данные. Если конфликтующие неатомарные операции не упорядочены через `happens-before`, получается `data race`.

`memory_order_relaxed` даёт только атомарность конкретной операции, но не создаёт синхронизации. `memory_order_release` публикует все действия, которые были до него в потоке. `memory_order_acquire` принимает эту публикацию, если прочитал значение из `release`-операции. Вместе `release/acquire` создают `synchronizes-with` и через него `happens-before`. `memory_order_acq_rel` используется для `read-modify-write` операций, а `memory_order_seq_cst` — самый сильный порядок, который добавляет единый глобальный порядок всех `seq_cst` операций.

Барьеры памяти ограничивают переупорядочивание операций памяти. В C++ есть `std::atomic_thread_fence`. На x86 пример аппаратного полного барьера — `mfence`. Но в обычном C++ коде лучше использовать `std::atomic` и нужный memory order, а не писать `mfence` вручную: компилятор сам отобразит операции на инструкции конкретной архитектуры.

28. Мини-проверка

1. Что описывает модель памяти C++?
2. Что такое data race?
3. Чем data race отличается от race condition?
4. Что такое `sequenced-before`?
5. Что такое `synchronizes-with`?
6. Что такое `happens-before`?
7. Как release/acquire создают happens-before?
8. Что даёт `memory_order_relaxed`?
9. Что дают `memory_order_release` и `memory_order_acquire`?
10. Что такое `memory_order_seq_cst`?
11. Почему relaxed нельзя использовать для публикации обычных данных через флаг?
12. Что такое memory fence и зачем нужен `mfence`?